

Baseline-First Classification of a Mixed-Type Tabular Dataset (Variant 2)

Ta Le Hanh Tho - G2606332B

IN6227-2026-Assignment-1 | GitHub: [link](#)

LLM: Claude Sonnet 5 | Interface: Claude web/mobile chat with code execution (interface version unavailable in this runtime) | Skill version: 2.7.1

INTRODUCTION

This report builds a classifier for the binary target *label* (yes/no) from 15 mixed features (8 categorical, 7 numeric) using *train.csv* (31,112 rows) and *test.csv* (13,334 rows). The workflow is baseline-first: a simple model is fitted and interpreted as evidence, and a more complex model class is added only if that evidence gives a reason. The primary metric is average precision (AP), a summary of the precision-recall curve, for the minority class “yes” (24.0% of labelled training rows).

METHODS OR PROCEDURES

1. Data exploration and cleaning. Three training rows and one test row lacked a label. Labels are never imputed, so these rows were excluded from fitting and from metric calculation respectively (0.01%, far below a 5% concern level), leaving 31,109 training and 13,333 evaluable test rows. Feature missingness was only 2–7 values per column (<0.03%) and there were no duplicate rows, so no further rows were removed and in-pipeline imputation sufficed. No whitespace or case variants were found in categories. “Other” and “Unknown” were kept as genuine levels because their positive rates differ sharply (22% vs 9% in *personal_interest*), so merging them, or treating them as missing, would discard signal. Apparent value caps (e.g., *aptitude_score* = 510, *performance_score* = 1010) could not be shown to be errors, so nothing was clipped.

2. Preprocessing. All transforms sit inside a scikit-learn Pipeline/ColumnTransformer fitted on training folds only. For Logistic Regression (LR), numerics used median imputation (preferred to the mean because *load_ratio* and *index_weight* are right-skewed, skew 1.7 and 1.5, with maxima far above their 99th percentiles) and standardisation (L2 regularisation penalises coefficients on the feature scale). Categoricals were mode-imputed and one-hot encoded (82 columns in total), with levels under 20 rows pooled as “infrequent” (region has 21 levels; mineral_type “Chalk” has 47 rows) so that tiny levels cannot receive unstable coefficients and unseen levels are handled. A log transform of skewed columns was not tested. For HistGradientBoosting (HGB), numerics were left unscaled (tree splits are invariant to monotone scaling and HGB handles NaN natively) and categoricals were ordinal-encoded and declared through *categorical_features*, because plain integer codes would otherwise be split as ordered numbers. This was verified (last 8 of 15 columns flagged; maximum cardinality 21, below the 255-bin limit; unseen levels map to NaN) on scikit-learn 1.8.0 (scikit-learn developers, 2026b).

3. Feature selection/engineering. No feature was removed and none was added. No identifier or target-derived column exists. *composite_rank* correlates 0.88 with *performance_score* and 0.83 with *stability_index*, which suggests a derived summary, but it is also a strong class separator (median 3.76 for yes vs 2.99 for no) and collinearity does not invalidate a regularised or tree model, so it was retained (a removal ablation was not run). No engineered feature was added because the baseline error analysis (step 5) pointed to no specific missing transformation.

4. Train/validation/test protocol. The files were interpreted as Train + Test because their schemas are identical. *test.csv* was read before model selection only for structural compatibility (columns, row count, parseability). No test values were used for fitting, preprocessing, feature selection, model selection, tuning or threshold choice; the test set was scored once, after models were frozen. With no validation file, stratified 5-fold CV (shuffled, seed 42) on the training data was the validation mechanism; stratification preserves the 24% positive rate per fold. Because the same CV compared and tuned models, CV scores are model-selection estimates that can be optimistic (Varma & Simon, 2006). Nested CV was not used because an independent test set exists and it would multiply the cost.

5. Candidate model selection. *Baseline.* LR was the starting point because (i) 82 dense columns for 31,109 rows is comfortably within its regime; (ii) class medians and positive rates move monotonically with several features (*composite_rank*, *aptitude_score*, *brew_preference*), making a linear log-odds hypothesis plausible; and (iii) its regularised, inspectable coefficients are documented (scikit-learn developers, 2026a). A single decision tree, the closest alternative, was not used because it is high-variance on 21-level categoricalals and could not tell us whether the linear hypothesis was adequate.

What the baseline revealed. Tuned LR ranked well (CV AP 0.709, ROC-AUC 0.891) but recovered only 57% of positives (out-of-fold: 3,239 missed vs 1,799 false alarms). Errors concentrated where classes mix: across *composite_rank* quintiles, error rose from 1.7% to 24.8% as the positive rate rose from 1.7% to 44%, and interests with 45–48% positives (Astronomy, Falconry) had 21–23% error.

Escalation. Because a linear model cannot express interactions among the eight categoricalals or threshold effects in the numerics, it was unclear whether this overlap was irreducible or a missed interaction. One nonlinear class, HGB, was tested to discriminate between the two: it splits categoricalals natively and suits 31k rows. Random forest, SVC, KNN, LinearSVC, Naive Bayes and CatBoost were not tested: the data are neither sparse nor high-dimensional (removing the advantage of LinearSVC/Naive Bayes), no observation motivated them over HGB, and kernel/distance methods add cost on one-hot data.

Retention rule (fixed after CV, before test scoring): keep HGB only if its paired AP gain over LR is positive in every fold and large relative to the paired-difference variability. The gains were +0.016, +0.014, +0.001, +0.013, +0.011 (mean +0.011, SD

0.006, about 4 standard errors), so the rule was met, although the gain is small (below one between-fold SD of either model's AP, 0.014–0.016) and one fold gained only 0.001. HGB was retained; escalation stopped because ~0.01 AP from adding nonlinearity and interactions suggests the remaining overlap is mostly not recoverable from these features.

6. Hyperparameter tuning. LR: a 10-configuration grid, $C \in \{0.01, 0.1, 1, 10, 100\} \times \text{class_weight} \in \{\text{none}, \text{balanced}\}$, was enough because a full 5-fold CV took about 1.5 s. AP plateaued for $C \geq 1$ (0.709) and fell to 0.698 at $C = 0.01$, so only strong shrinkage under-fits; balanced weighting lowered AP by about 0.003 at every C and was not used (it moves the operating point, not the ranking, and the 0.5 threshold was left fixed). HGB: a 10-trial randomised search (50 fits, about 33 s on one CPU, a light-to-moderate workload needing no reduction) over learning_rate, max_iter, max_leaf_nodes, l2_regularization and min_samples_leaf (Table 2), with early stopping (10% internal validation, patience 20). The ten trials scored only 0.716–0.720 AP, so a larger search was not expected to change the conclusion.

7. Evaluation metrics. AP is primary because the 76/24 imbalance makes accuracy uninformative (always predicting “no” scores 76%) and ROC-AUC less sensitive to minority-class precision. ROC-AUC, F1 (yes), macro-F1 and balanced accuracy are secondary. Label-based metrics use a fixed 0.5 threshold; calibration and threshold tuning were not assessed.

RESULTS

Table 1. Model comparison (CV = 5-fold model-selection estimate, mean $\pm$ SD; test = single final evaluation, 13,333 rows, 23.8% positive).

Model	CV AP	Test AP	Test ROC-AUC	Test F1 (yes)	Test macro-F1	Test bal. acc.
LR baseline ($C = 100$)	0.709 $\pm$ 0.014	0.702	0.889	0.628	0.763	0.747
HGB (retained)	0.720 $\pm$ 0.016	0.708	0.891	0.636	0.767	0.753

Table 2. Selected hyperparameters (other settings scikit-learn defaults; seed 42).

Model	Selected values
LR	$C = 100$, class_weight = none, L2 penalty, max_iter = 2000
HGB	learning_rate = 0.03, max_iter = 200 (early stopping never triggered), max_leaf_nodes = 15, min_samples_leaf = 50, l2_regularization = 0

Per class on the test set, HGB scored precision/recall/F1 of 0.877/0.920/0.898 for “no” and 0.695/0.586/0.636 for “yes” (LR: 0.873/0.923/0.898 and 0.698/0.570/0.628). Confusion counts (TN/FP/FN/TP) were 9,351/814/1,312/1,856 for HGB and 9,384/781/1,362/1,806 for LR. A paired bootstrap on the test rows (1,000 resamples) gave an AP gain of +0.006 for HGB (95% interval 0.0005 to 0.0115).

DISCUSSION

LR was the simplest defensible start, and its result was informative: ranking quality was good, but recall for “yes” was weak and errors sat in regions where the classes genuinely mix. HGB tested whether interactions explained this. It helped only slightly: CV AP rose by about 0.011 and test AP by 0.006, with the bootstrap interval barely excluding zero, so the gain is real but small and should not be called decisive. The test gain is smaller than the CV gain and both models scored about 0.01 lower on test than in CV, consistent with model-selection optimism and sampling noise. HGB's benefit came from finding 50 more positives (1,856 vs 1,806) at a cost of 33 more false alarms; about 41% of positives were still missed. HGB was selected on the pre-test criterion of consistent paired CV gains; LR remains a defensible choice where inspectable coefficients or cost matter (measured: LR CV about 1.5 s vs about 33 s for the HGB search), and the evidence does not force a single winner.

Limitations: the HGB search and the paired comparison reused the same folds, so the CV gain is somewhat optimistic; only two model classes were tested; a *composite_rank* ablation and log transforms were not evaluated; and no threshold or calibration analysis was done, though a threshold chosen from out-of-fold predictions is the most promising route to better minority recall.

CONCLUSION

On 31,109 labelled training rows, a tuned Logistic Regression baseline (test AP 0.702) was only modestly improved by HistGradientBoosting with native categorical splits (test AP 0.708; macro-F1 0.767). Consistent but small gains from added nonlinearity suggest that most remaining error is overlap in the available features rather than model capacity, so further model complexity is unlikely to pay off before threshold or feature-level work.

REFERENCES

- scikit-learn developers. (2026a). *LogisticRegression*.
https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
- scikit-learn developers. (2026b). *HistGradientBoostingClassifier*.
<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.HistGradientBoostingClassifier.html>
- Varma, S., & Simon, R. (2006). Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics*, 7, Article 91. <https://doi.org/10.1186/1471-2105-7-91>